Lab 02: Advanced Abstraction, Encapsulation, Cohesion, Coupling, Stable Interfaces & Git

1. Learning Objectives

After completing this lab, you should be able to:

- Implement advanced abstraction using interfaces and abstract classes.
- Apply encapsulation to protect object state.
- Apply information hiding by hiding implementation details.
- Identify functional and coincidental cohesion.
- Analyze tight and loose coupling between components.
- Design stable interfaces that can evolve without unnecessarily affecting clients.
- Use Git branches for independent feature development.
- Merge a completed feature into the main development branch.
- Understand a basic collaborative Git workflow.

Scenario: Food Delivery System

We want to develop a small food delivery application.

The application should allow a customer to:

- select food items,
- create an order,
- calculate the order total,
- apply a payment method,
- generate an order summary.

We will initially create a simple implementation and then improve it using software construction principles.

Part 1 — Initial Implementation

Start with a simple version.

```
public class FoodDeliveryApp {
    public static void main(String[] args) {
        String customerName = "Ali";
        String item1 = "Burger";
        double price1 = 500;
        String item2 = "Pizza";
        double price2 = 900;
        double total = price1 + price2;
        System.out.println("Customer: " + customerName);
        System.out.println("Item: " + item1);
        System.out.println("Price: " + price1);
        System.out.println("Item: " + item2);
        System.out.println("Price: " + price2);
        System.out.println("Total: " + total);
    }
}
```

This program works, but the design is poor.
Everything is inside: main()
The method is responsible for:

- storing customer information,
- storing food information,
- calculating prices,
- handling the order,
- displaying the order.

This gives us an opportunity to apply the concepts one by one.

Part 2 — Abstraction

What is abstraction?

Abstraction means exposing the important behavior while hiding unnecessary implementation details.
For example, the application should not need to know **how** an order calculates its total.
It should simply ask:

```
order.calculateTotal();
```

The calculation details remain inside the order.

3. Create a FoodItem Abstraction

Create:

```
public class FoodItem {
   private String name;
   private double price;
   public FoodItem(String name, double price) {
      this.name = name;
      this.price = price;
   }
   public String getName() {
      return name;
   }
   public double getPrice() {
      return price;
   }
}
```

Now instead of:

```
String item1 = "Burger";
double price1 = 500;
```

we can write:

```
FoodItem burger = new FoodItem("Burger", 500);
```

And:

```
FoodItem pizza = new FoodItem("Pizza", 900);
```

The concept of a food item is now represented by an object.

4. Encapsulation

Notice:

```
private String name;
private double price;
```

The data is private.

This prevents other classes from directly modifying it.

For example, this is not allowed:

```
burger.price = -500;
```

because price is private.

Instead, access is controlled through:

```
burger.getPrice();
```

This is encapsulation.

5. Improve Encapsulation

However, our constructor currently accepts:

```
new FoodItem("Burger", -500);
```

That would create an invalid food item.

We should protect the object's state.

```
public class FoodItem {
   private String name;
   private double price;
   public FoodItem(String name, double price) {
      if (name == null || name.isBlank()) {
         throw new IllegalArgumentException(
              "Food name cannot be empty." ); }
      if (price < 0) {
         throw new IllegalArgumentException(
              "Price cannot be negative." ); }
                this.name = name;
                this.price = price;}
              public String getName() {
                return name;
              }
              public double getPrice() {
                return price;
              }
            }
```

Now the class protects its own state.

For example:

```
FoodItem burger = new FoodItem("Burger", 500);
```

is valid.

But:

```
FoodItem burger = new FoodItem("Burger", -500);
```

throws an exception.

6. Information Hiding

Encapsulation protects the object's state.

Information hiding goes one step further.

Suppose the application needs the price.

It uses: `burger.getPrice();`

It does **not** need to know how the price is stored internally.

Today: `private double price;`

Tomorrow we could change the internal representation.

For example:

```
private int priceInCents;
```

The rest of the application should not need to know.

The public interface remains: `getPrice()`

Therefore:

The internal representation is hidden from the client.

Part 3 — Order Abstraction

Now create an Order.

```
import java.util.ArrayList;
import java.util.List;
public class Order {
   private List<FoodItem> items;
   public Order() {
      items = new ArrayList<>();
   }
   public void addItem(FoodItem item) {
      items.add(item);
   }
   public double calculateTotal() {
      double total = 0;
      for (FoodItem item : items) {
         total += item.getPrice();
      }
      return total;
   }
}
```

Now the application can simply say:

```
order.calculateTotal();
```

The caller does not need to know:

```
double total = 0;
for (FoodItem item : items) {
   total += item.getPrice();
}
```

That calculation is hidden inside Order.

7. Information Hiding in Order

Notice:

```
private List<FoodItem> items;
```

The list is private.

We don't expose it directly.

This prevents external code from doing something like:

```
order.items.clear();
```

because items are hidden.

Instead, the order controls its own operations:

```
order.addItem(burger);
order.addItem(pizza);
```

This is good encapsulation and information hiding.

Part 4 — Functional Cohesion

Now look at the Order class:

```
public class Order {
   private List<FoodItem> items;
   public void addItem(FoodItem item) {
      items.add(item);
   }
   public double calculateTotal() {
   }
}
```

The operations are related to one clear responsibility:

Managing an order.

This is **functional cohesion**.

The methods:

```
addItem()
calculateTotal()
```

work together to manage the order.

8. Example of Poor / Coincidental Cohesion

Imagine instead that we create:

```
public class FoodSystem {
  public void addItem() {
    // add food }
  public double calculateTotal() {
    // calculate total
    return 0; }
  public void sendEmail() {
    // send email }
  public void saveToDatabase() {
    // save database }
  public void printReceipt() {
    // print receipt }
  public void calculateDeliveryDistance() {
    // calculate distance } }
```

This class contains unrelated responsibilities.

It handles:

- order management,
- email,
- database,
- printing,
- delivery calculation.

These activities have little relationship.

This is an example of **coincidental cohesion**.

9. Improve the Cohesion

Separate the responsibilities.

```
Order
  ├── addItem()
  └── calculateTotal()
EmailService
  └── sendEmail()
OrderRepository
  └── save()
Receipt
  └── print()
DeliveryCalculator
  └── calculateDistance()
```

Now each class has a focused responsibility.

This gives us stronger functional cohesion.

Part 5 — Abstraction with Payment

Now suppose the application supports:

- credit card payment,
- PayPal payment,
- cash payment.

If we write everything directly in Order, we will eventually have a large number of if-else statements.
Instead, create an abstraction.

```
public interface PaymentMethod {
   void pay(double amount);
}
```

This is a stable abstraction.

It says:

Any payment method must provide a pay() operation.

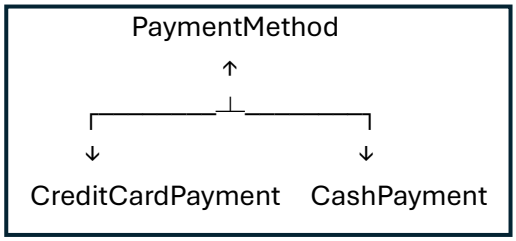
10. Credit Card Implementation

```
public class CreditCardPayment
      implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using credit card." );
   }
}
```

11. Cash Payment

```
public class CashPayment
      implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println(
            "Paid " + amount +
            " using cash."
      );
   }
}
```

Now we have:



Part 6 — Tight Coupling

Consider this design:

```
public class OrderService {
   private CreditCardPayment payment;
   public OrderService() {
      payment = new CreditCardPayment();
   }
   public void checkout(Order order) {
      double total =
            order.calculateTotal();
      payment.pay(total);
   }
}
```

The problem is:

```
new CreditCardPayment();
```

OrderService is directly dependent on CreditCardPayment.

If we want cash payment, we must modify OrderService.

This is **tight coupling**.

13. Loose Coupling

Instead, depend on the interface:

```
public class OrderService {
   private PaymentMethod payment;
   public OrderService(PaymentMethod payment) {
      this.payment = payment;
   }
   public void checkout(Order order) {
      double total = order.calculateTotal();
      payment.pay(total);
   }
}
```

Now OrderService does not care which payment method is being used.

It only knows: PaymentMethod

14. Using Different Implementations

Credit card:

```
PaymentMethod payment = new CreditCardPayment();
OrderService service = new OrderService(payment);
```

Cash:

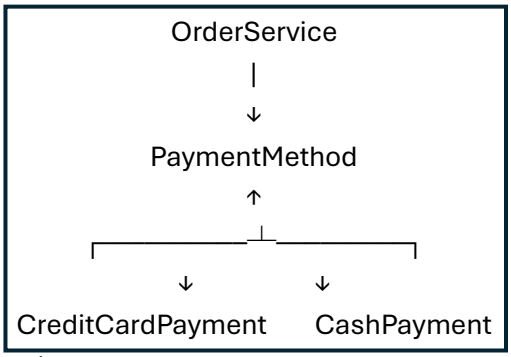
```
PaymentMethod payment = new CashPayment();
OrderService service = new OrderService(payment);
```

The OrderService class does not change.

This is **loose coupling**.

15. Why This Is Better?

Our design now looks like:



The service depends on the abstraction.

It does not depend directly on the particular payment implementation.

Part 7 — Stable Interfaces

The interface:

```
public interface PaymentMethod {
  void pay(double amount);
}
```

is a contract.

The client only needs to know: `payment.pay(amount);`

It doesn't need to know whether payment is processed using:

- a credit card,
- cash,
- PayPal,
- a mobile wallet,
- another payment provider.

For example, we can later add:

```
public class MobileWalletPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
        "Paid " + amount +
        " using mobile wallet.");
  }
}
```

No modification is required in: OrderService

We simply provide another implementation.

16. What Makes the Interface Stable?

A stable interface should expose the **essential behavior**.

Good:

```
public interface PaymentMethod {
    void pay(double amount);
}
```

Poor design would expose implementation-specific details:

```
public interface PaymentMethod {
    void connectToBankServer();
    void encryptCardNumber();
    void createTransactionPacket();
    void sendHTTPRequest();
    void pay(double amount);
}
```

Why is this bad?

Because those operations are implementation details.

A cash payment does not need: `connectToBankServer()`

The interface has become dependent on one particular implementation.

A stable interface should avoid exposing unnecessary implementation details.

Part 8 — Complete Connected Example

Now let's put the concepts together.

FoodItem.java

```
public class FoodItem {
    private String name;
    private double price;
    public FoodItem(String name, double price) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException(
                "Food name cannot be empty." );
        }
        if (price < 0) {
            throw new IllegalArgumentException(
                "Price cannot be negative." ); }
        this.name = name;
        this.price = price; }
    public String getName() {
        return name; }
    public double getPrice() {
        return price;
    }
}
```

Order.java

```
import java.util.ArrayList;
import java.util.List;
public class Order {
   private List<FoodItem> items;
   public Order() {
      items = new ArrayList<>();
   }
   public void addItem(FoodItem item) {
      if (item == null) {
         throw new IllegalArgumentException(
               "Item cannot be null." );
      }
      items.add(item);
   }
   public double calculateTotal() {
      double total = 0;
      for (FoodItem item : items) {
         total += item.getPrice();
      }
      return total; } }
```

PaymentMethod.java

```
public interface PaymentMethod {
   void pay(double amount);
}
```

CreditCardPayment.java

```
public class CreditCardPayment
      implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using credit card." );
   }
}
```

CashPayment.java

```
public class CashPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println("Paid " + amount + " using cash." );
  }
}
```

OrderService.java

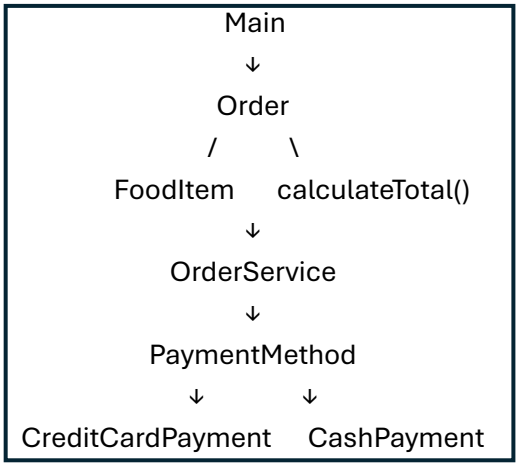
```
public class OrderService {
  private PaymentMethod payment;
  public OrderService(PaymentMethod payment) {
    this.payment = payment;
  }
  public void checkout(Order order) {
    double total = order.calculateTotal();
    payment.pay(total);
    System.out.println("Order completed." );
  }
}
```

Main.java

```
public class Main {
  public static void main(String[] args) {
    FoodItem burger = new FoodItem("Burger", 500 );
    FoodItem pizza = new FoodItem("Pizza", 900 );
    Order order = new Order();
    order.addItem(burger);
    order.addItem(pizza);
    PaymentMethod payment = new CreditCardPayment();
    OrderService service = new OrderService(payment);
    service.checkout(order);
  }
}
```

Part 9 — How the Concepts Are Connected

The final design is:



Now map the concepts.

1. Advanced abstraction

Interfaces represent behavior: PaymentMethod

The client knows **what** payment can do, not **how** it does.

2. Encapsulation

FoodItem protects:

```
private String name;
private double price;
```

Order protects:

```
private List<FoodItem> items;
```

3. Information hiding

The internal implementation of: `calculateTotal()`

is hidden.

The caller only uses: `order.calculateTotal();`

Similarly, the payment implementation is hidden behind:

`payment.pay(amount);`

4. Functional cohesion

Order focuses on order-related operations.

OrderService focuses on checkout.

Each class has a meaningful responsibility.

5. Coincidental cohesion

We avoid creating one huge class containing:

- order management
- payment
- email
- database
- receipt printing
- delivery calculation

6. Tight coupling

Avoid: `private CreditCardPayment payment;`

and:

`payment = new CreditCardPayment();`

inside OrderService.

7. Loose coupling

Use: `private PaymentMethod payment;`

and inject the implementation: `new OrderService(payment);`

8. Stable interface

Clients depend on: `PaymentMethod`

rather than: `CreditCardPayment`

New payment implementations can be introduced without changing OrderService.

Part 10 — Git Branching

Now connect Git with the same project.

Suppose the current version supports:

- Burger
- Pizza
- Credit Card
- Cash

The team receives a new requirement:

Add mobile wallet payment.

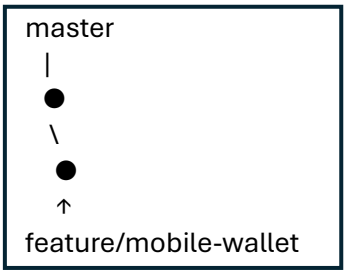
Instead of directly changing the stable master branch, create a feature branch.

First check: **git branch**

Suppose: *** master**

Create the branch: **git switch -c feature/mobile-wallet**

Now:



We work on the feature branch.

17. Implement the New Feature

Create:

```
public class MobileWalletPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println( "Paid " + amount +" using mobile wallet." );
  }
}
```

Change:

```
PaymentMethod payment = new CreditCardPayment();
```

to:

```
PaymentMethod payment = new MobileWalletPayment();
```

Test the application.

18. Commit the Feature

Check: **git status**

Review: **git diff**

Stage: **git add .**

Commit: **git commit -m "Add mobile wallet payment"**

Now the feature is safely saved on: **feature/mobile-wallet**

19. Return to Master

After testing: **git switch master**

The stable branch remains unchanged.

Then merge: **git merge feature/mobile-wallet**

Now the mobile wallet feature becomes part of master.

20. Delete the Feature Branch

After successful merging: **git branch -d feature/mobile-wallet**

21. Complete Git Workflow

Stable Software
↓
master branch
↓
New mobile-wallet feature
↓
git switch -c feature/mobile-wallet
↓
Write the code
↓
Test
↓
git add .
↓
commit

↓

git switch master

↓

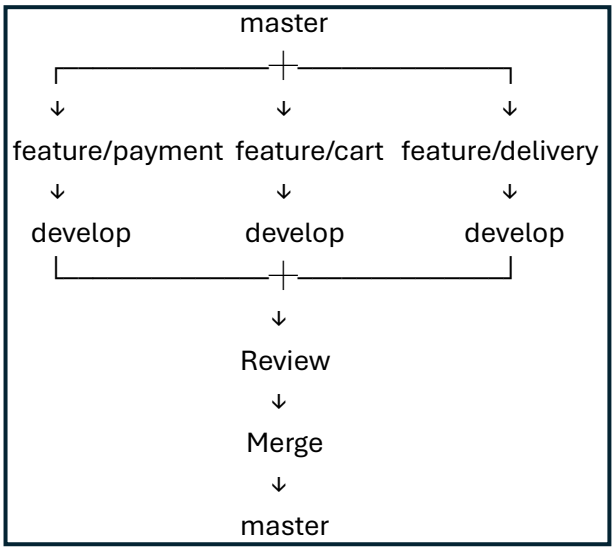
git merge feature/mobile-wallet

↓

Updated master branch

22. Basic Collaborative Workflow

In a team, different features can be developed independently:



A basic workflow is: **git pull**

Create a feature branch: **git switch -c feature/mobile-wallet**

Develop and test.

Then:

git status

git diff

git add .

git commit -m "Add mobile wallet payment"

Share the branch: **git push -u origin feature/mobile-wallet**

After review:

git switch master

git pull

git merge feature/mobile-wallet

Then: **git push**